Two-Dimensional FFT on Clusters

Isaac D. Scherson (aka The Schark )

Dept. of Computer Science (Systems)

School of Information and Computer Sciences

University of California, Irvine

Irvine, CA 92697-3425

isaac@ics.uci.edu

www.ics.uci.edu/ ˜ isaac www.ics.uci.edu/ ˜ schark

Preliminary Assumptions

Jean Baptiste Joseph Fourier (1768-1830)

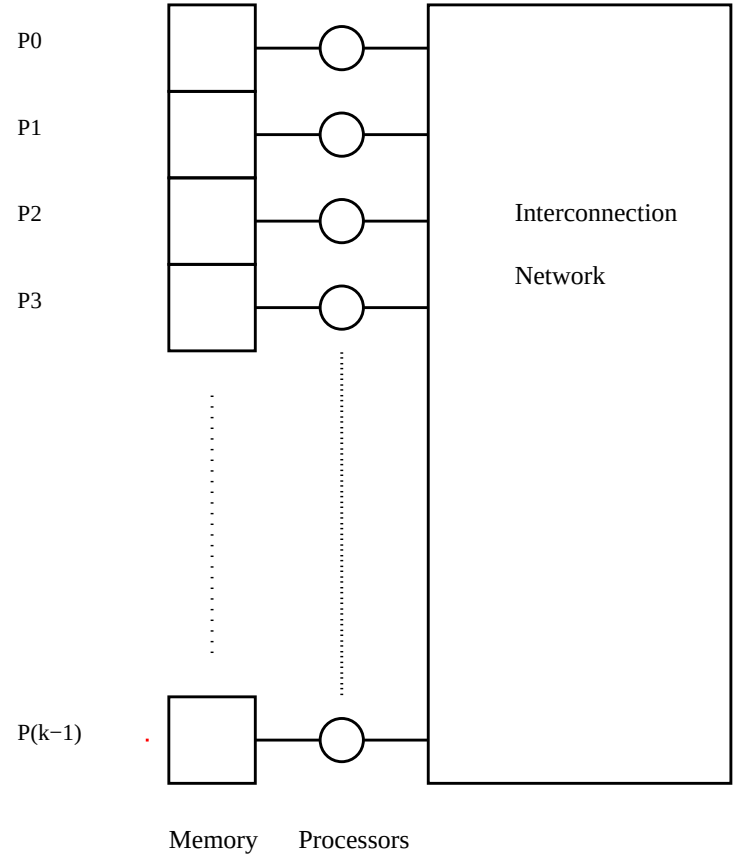
We assume basic knowledge of the Fourier Transform and its variations: the DFT and its implementation as FFT.

- Cooley and Tukey's algorithm. Decimation in time or frequency.
- If necessary, pause to remind basic FFT algorithm. Use PPT file fft_intro.ppt

Problem Statement

- Consider the problem of computing the 2D-FFT of the $n \times n$ complex matrix A
- The problem is to be solved on a k-processor Cluster

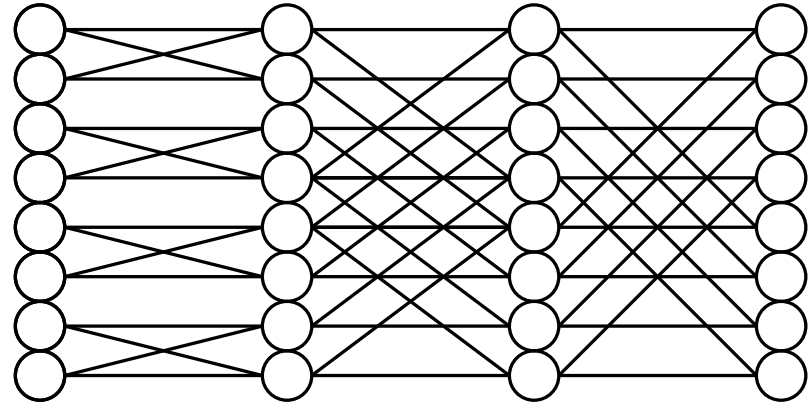
$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0(n-1)} \\ a_{10} & a_{11} & \cdots & a_{1(n-1)} \\ \vdots & & & \vdots \\ a_{(n-1)0} & a_{(n-1)1} & \cdots & a_{(n-1)(n-1)} \end{bmatrix}$$



2D-FFT

- 2D FFT is based on the 1D FFT
- FFT of rows followed by FFT of columns

- Basic Procedure:
  - FFT rows
  - Transpose
  - FFT rows
  - Transpose



An example of the data flow for an eight-point FFT

Data Distribution

- Each processor P_i is given a band of $\frac{n}{k}$ rows of the matrix starting with row $i \times \frac{n}{k}$

$$k\ bands \left\{ \begin{bmatrix} \boxed{B_{00}} & \boxed{B_{01}} & \boxed{B_{02}} & \cdots & \boxed{B_{0(k-1)}} \\ \boxed{B_{10}} & \boxed{B_{11}} & \boxed{B_{12}} & \cdots & \boxed{B_{1(k-1)}} \\ \boxed{B_{20}} & \boxed{B_{21}} & \boxed{B_{22}} & \cdots & \boxed{B_{2(k-1)}} \\ \vdots & & & & \\ \boxed{B_{(k-1)0}} & \boxed{B_{(k-1)1}} & \boxed{B_{(k-1)2}} & \cdots & \boxed{B_{(k-1)(k-1)}} \end{bmatrix} \right.$$

- Each block B_{ij} is a square block of $\frac{n}{k} \times \frac{n}{k}$ elements of A

Implementation of 2D FFT

- Eack processor P_i computes $\frac{n}{k}$ 1D-FFTs on the $n - element$ row vectors in its memory

- The problem now is how to transpose ...

  - Note that the transpose of A is obtained by transposing the blocks

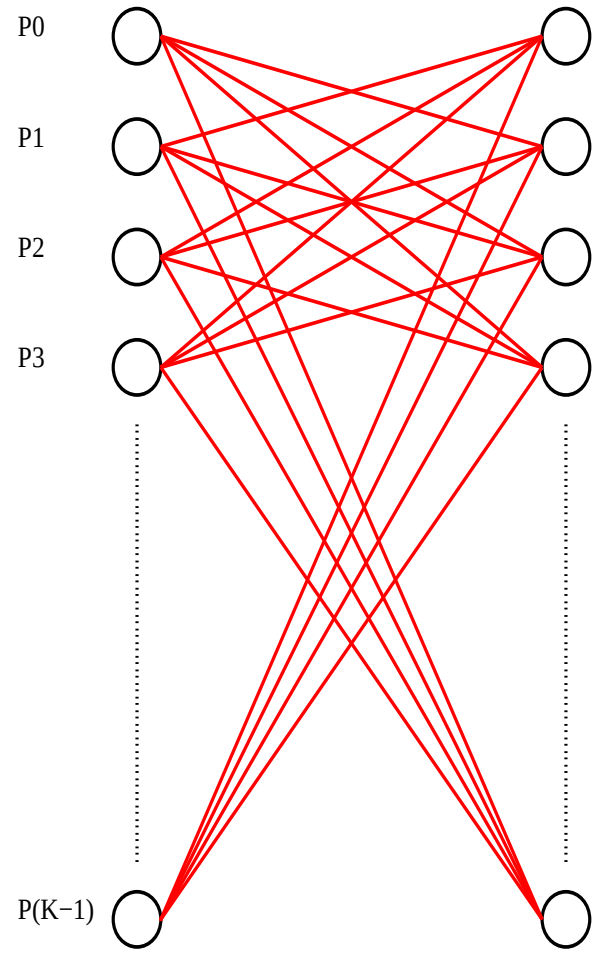
Transposing by Blocks

- Observations:
  - All Processors need to distribute all, but one, of their blocks among all other processors:
    - Processor P_i sends Block B_{ij} to Processor P_j for all $j \in [0, k-1] - [i]$

- Basic Sequential Loop:

```
For i = 0, k − 1, do:
    For j = 0, k − 1, do:
        P_i sends Block B_ij
         to Processot P_j
    endfor
endfor
```

For simplicity, the loop does not optimize for a Processor sending to itself



Graph of All Required Communication Patterns

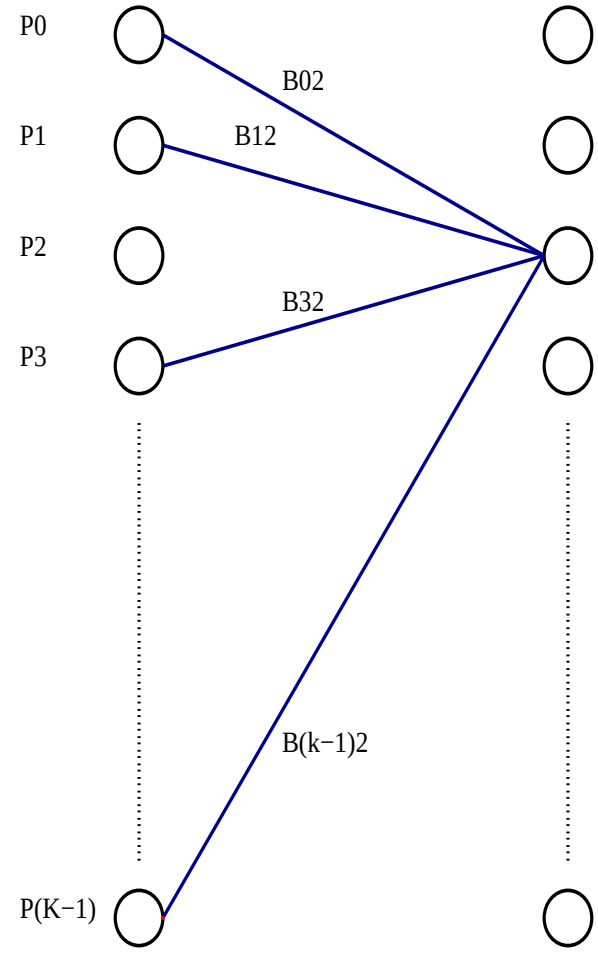
Complexity of Transfer = O(k^2) block transfers

Parallelizing on i

- Choosing to transfer according to the loop:

```
For all i, do parallel:
      For j = 0, k − 1, do:
          P_i sends Block B_ij
           to Processot P_j
      endfor
endfor
```

Complexity of Transfer = O(k^2) block transfers

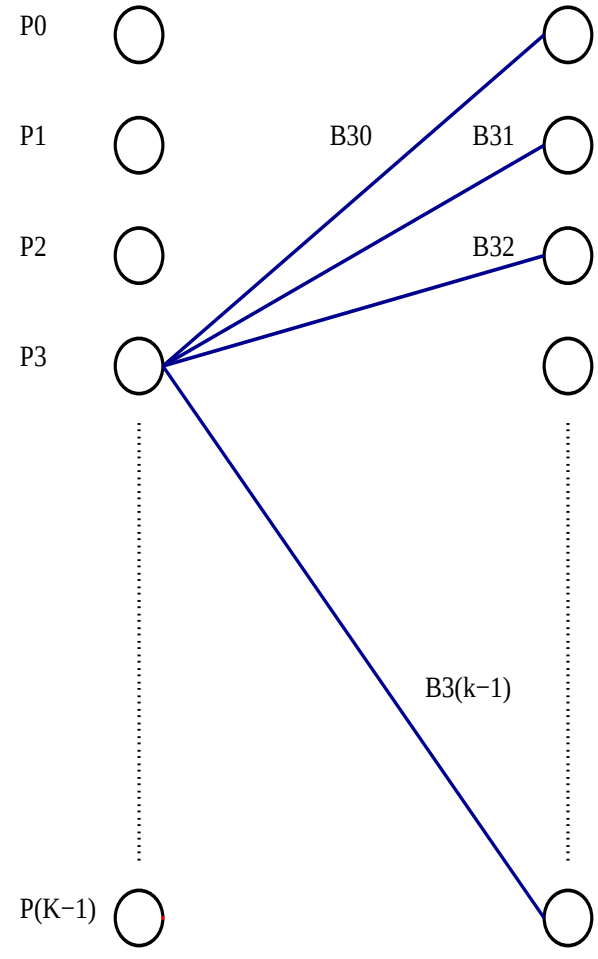


Parallelizing on j

- Choosing to transfer according to the loop:

```
For i = 0, k − 1, do:
      For all j, do parallel:
          P_i sends Block B_ij
           to Processot P_j
      endfor
endfor
```

Complexity of Transfer = O(k^2) block transfers



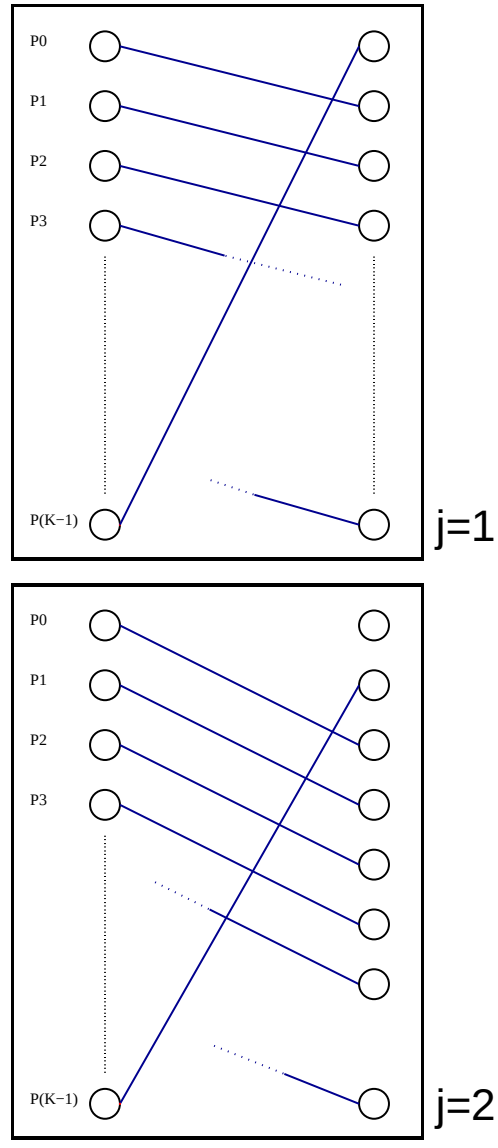
Astute Parallelizing on i

- Choosing to transfer according to the loop:

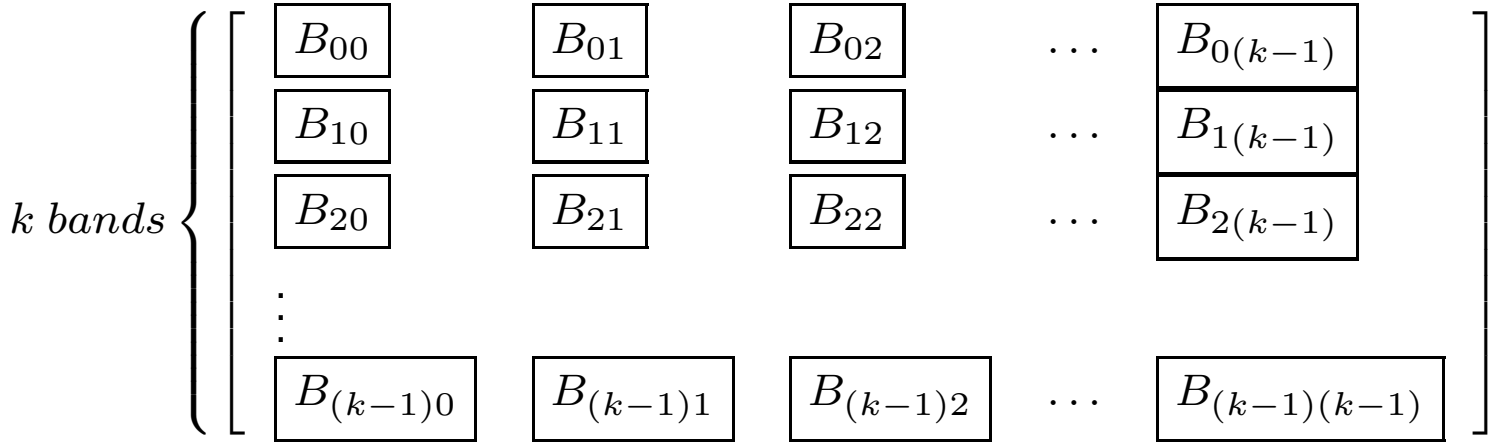
For all i, do parallel:
    For all $j = 1, k-1$, do:
        P_i sends Block $B_{i(j+i)modk}$
        to Processot $P_{(j+i)modk}$
    endfor
endfor

Complexity of Transfer = O(k) block transfers

If the network cooperates !!! ... That is, if it is a X-bar Switch.



Final Implementation Discussion



- Using MPI, the solution is trivial and takes advantage of the astute transpose mechanism.
- Using DSM, the solution takes advantage of the same principle and can overlap communications and computations by using the following two conditions:
  1. Use a double buffer to write into at the last iteration of the 1D-FFT
  2. At processor P_i, start iterations with butterfly anchored on element i of each row vector. This skews the vector writing and avoids the all to one problem.

Performance Analysis

- Each of the k processors performs $2 \times \frac{n}{k}$ 1D-FFts
  - This is a factor of $O(k)$ speedup with respect to the sequential case.
- In the sequential case, there is no need to transpose
  - The FFT of rows and columns are done by two separate loops that index over rows and columns respectively

HENCE

The Maximum Speedup one can expect, in the BEST POSSIBLE CASE:

$$Speedup = \frac{Time\ in\ Single\ Processor}{Time\ in\ Cluster}$$

$$= \frac{2 \times n^2 \times \log n}{2\frac{n^2}{k} \log n + O(k\ (\frac{n}{k} \times \frac{n}{k})\ block\ transfers)}$$

THE END

. . . QUESTIONS ?